Lecture 8: Parallel Programming and Architectures

Objective:
Introduce the fundamentals of parallel programming and architectures used in High-Performance Computing (HPC).

Duration: 1 Hour

🎯 Learning Outcomes

By the end of this lecture, students will be able to:

- Understand the need for parallelism in computing
- Differentiate between various types of parallelism
- Describe shared, distributed, and hybrid memory architectures
- Identify popular parallel programming paradigms like MPI, OpenMP, and CUDA

🧩 1. Parallel Programming Overview

📌 Why Parallel Programming?

- Solves **large-scale problems** faster by dividing tasks among multiple processors
- Used in simulations, deep learning, weather forecasting, genome sequencing, etc.
- Utilizes **multi-core processors** and **GPUs** effectively

---

🧠 Types of Parallelism

Type	Description	Example
Task Parallelism	Different tasks/processes run in parallel	Simultaneous data analysis and reporting
Data Parallelism	Same operation performed on chunks of data	Image processing (applying a filter on all pixels)

Pipeline Parallelism	Tasks divided into stages; each stage processed in parallel	Video streaming, instruction pipelines in CPUs

---

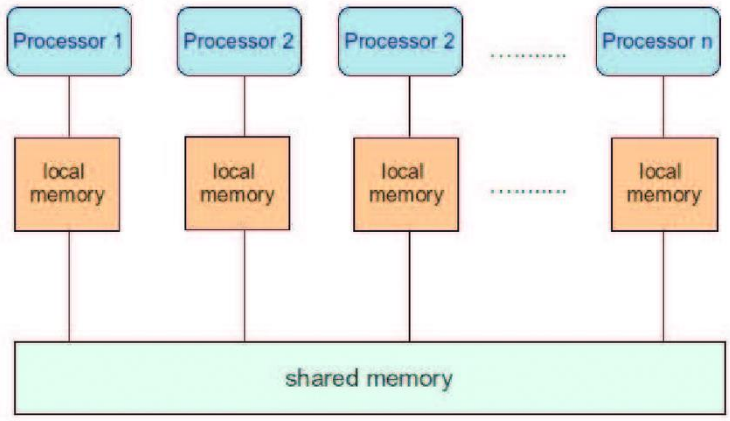
🧪 Simple Example: Data Parallelism

```
// Add two arrays using parallel loop (OpenMP)
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    c[i] = a[i] + b[i];
}
```

🖧 2. Parallel Architectures

🏠 a. Shared Memory Architecture

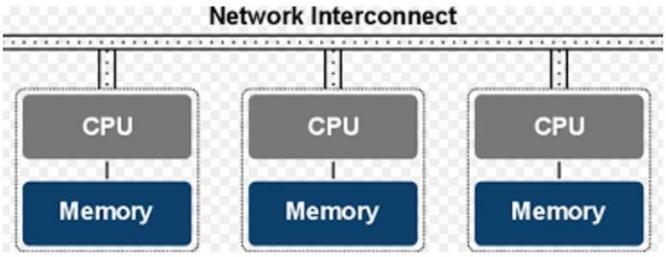
- All processors share a **common memory space**
- Fast communication, but prone to **memory conflicts**
- Example: Multi-core systems



Programming model: OpenMP

🌐 b. Distributed Memory Architecture

- Each processor has its **own local memory**
- Data exchange occurs via **message passing**
- Scalable and used in **clusters/supercomputers**

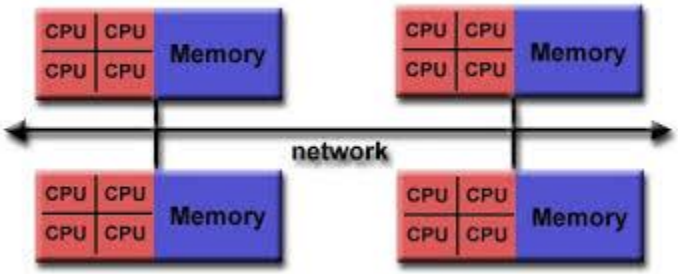


Distributed Memory

Programming model: MPI

♻️ c. Hybrid Architecture

- Combines shared and distributed memory models
- Example: Nodes with multiple cores (shared memory) connected across a cluster (distributed memory)



Programming model: MPI between nodes + OpenMP within nodes

🧠 **Architecture Summary Table**

Architecture	Memory Access	Communication Method	Used In
Shared Memory	Global memory	Through shared RAM	Laptops, Workstations
Distributed Memory	Local per processor	Message Passing	Clusters, Supercomputers
Hybrid	Mix of both	Mixed communication	Modern HPC systems

💻 **3. Parallel Programming Paradigms**

📦 **a. MPI (Message Passing Interface)**

- Standard for distributed memory systems
- Explicitly sends/receives data between processes
- Common functions: MPI_Init(), MPI_Send(), MPI_Recv(), MPI_Finalize()

Example Use Case: Weather simulation across compute nodes

📝 **b. OpenMP (Open Multi-Processing)**

- For shared memory systems (multi-core CPUs)
- Uses compiler directives like #pragma omp
- Easy to parallelize loops

Example Use Case: Parallel matrix multiplication on a desktop

---

🖼️ **c. CUDA (Compute Unified Device Architecture)**

- Developed by NVIDIA for **GPU-based** parallelism
- Allows massive data parallelism
- Uses kernel functions and blocks of threads

Example Use Case: Deep learning training on GPU

🏫 Classroom Activities

🔸 **Activity 1: Parallelism Spotting**

- Show real-life systems/tasks
- Ask students to identify the type of parallelism used

🔸 **Activity 2: Hands-On OpenMP (Lab optional)**

- Write a simple C program with and without OpenMP
- Compare runtime and output

🔸 **Activity 3: Group Discussion**

- Topic: "Will GPUs replace CPUs in future HPC systems?"

📚 Summary

Concept	Key Points
Parallel Programming	Needed for speed and scale
Types of Parallelism	Task, Data, Pipeline
Architectures	Shared, Distributed, Hybrid
Programming Paradigms	MPI (Distributed), OpenMP (Shared), CUDA (GPU)

📝 Homework/Assignment

Task:
Compare and contrast **MPI**, **OpenMP**, and **CUDA** under the following headings:

1. Type of memory architecture
2. Ease of programming

3. Use cases
4. Performance impact

Submission Format: Table or essay (300 words)
Deadline: Before next class